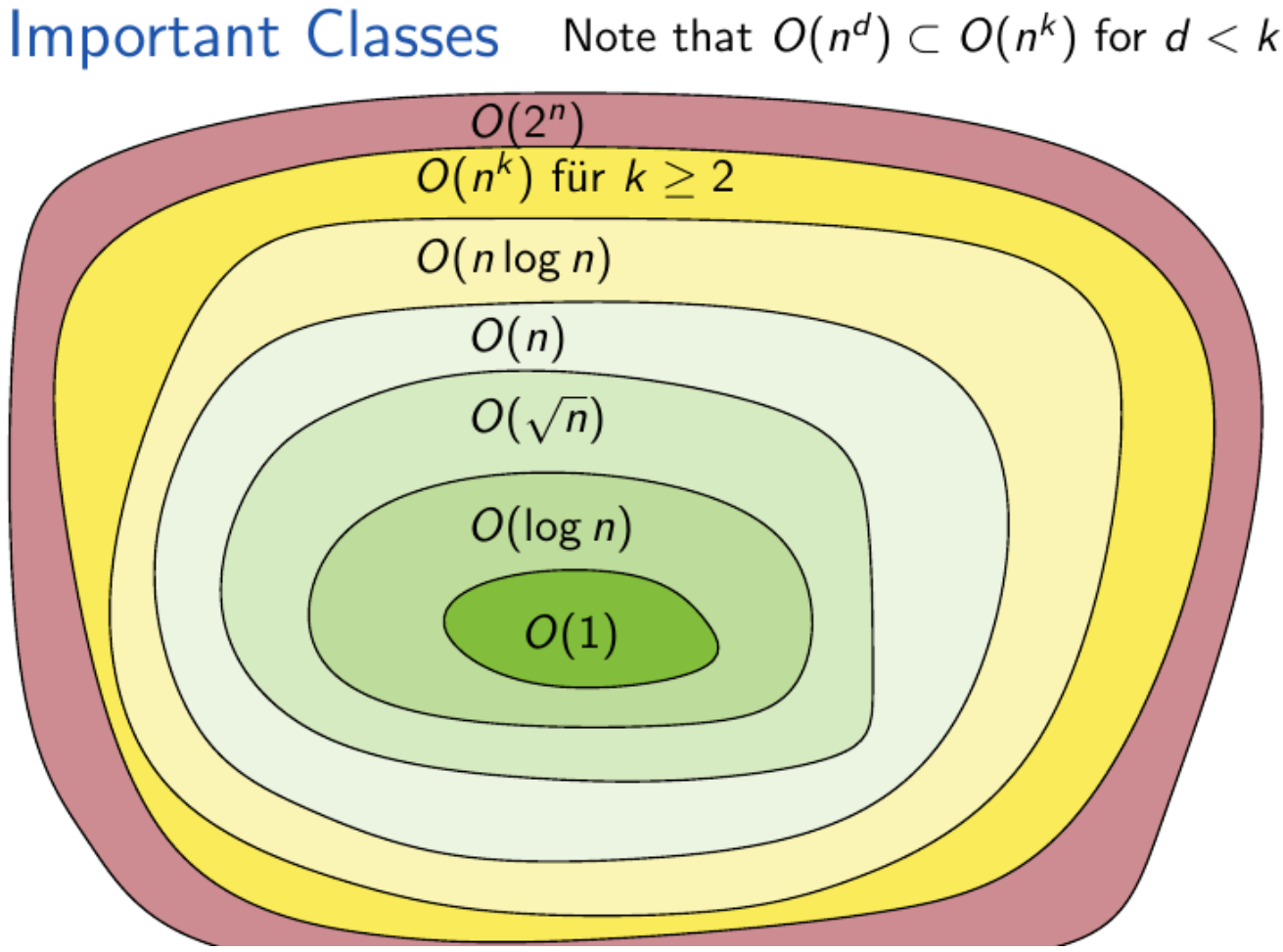
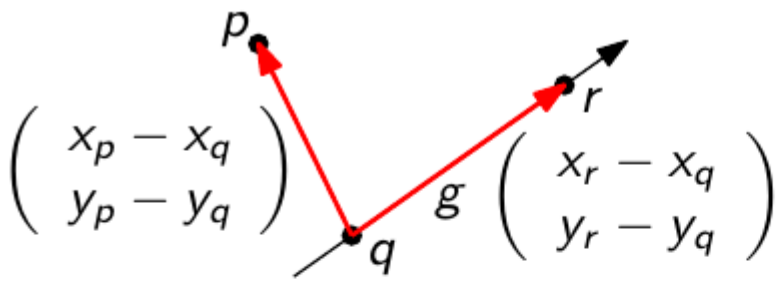
Lecture 4: Fundamental Geometric Algorithms

Big O notation

- Big O notation: tool to estimate growth of function (Runtimes, size of data structures)
- **Defintion**:
  - Let $g(n)$ be a function
  - $O(g(n))$ is a *set of functions*
  - Set $O(g(n))$ conatins a function $f(n)$ iff there exist constanc c and n_0, such that
    - $0 \leq f(n) \leq c \cdot g(n)$ for each $n \geq n_0$
- $f \in O(g)$ means : f grows at most as fast as g
- 

Point to Line Test



$$\det \begin{pmatrix} x_r - x_q & x_p - x_q \\ y_r - y_q & y_p - y_q \end{pmatrix} > 0 \Leftrightarrow \ p \text{ lies to the left of } g$$

$$\det \begin{pmatrix} x_r - x_q & x_p - x_q \\ y_r - y_q & y_p - y_q \end{pmatrix} = 0 \Leftrightarrow \ p \text{ lies on } g$$

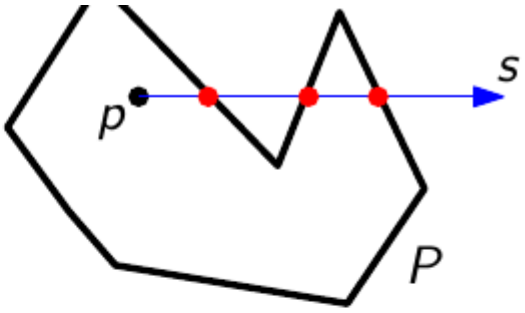
$$\det \begin{pmatrix} x_r - x_q & x_p - x_q \\ y_r - y_q & y_p - y_q \end{pmatrix} < 0 \Leftrightarrow \ p \text{ lies to the right of } g$$

- In $\mathcal{O}(1)$

Point in Polygon Test

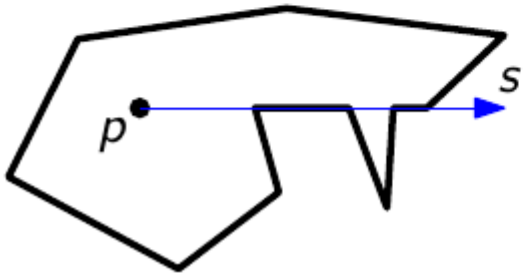
Idea: Count intersections of horizontal ray from p with polygon boundary.

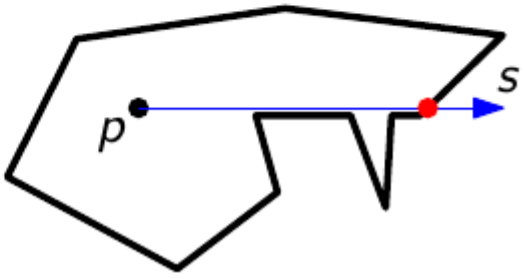
odd number $\Rightarrow$ p is within P
even number $\Rightarrow$ p is not within P



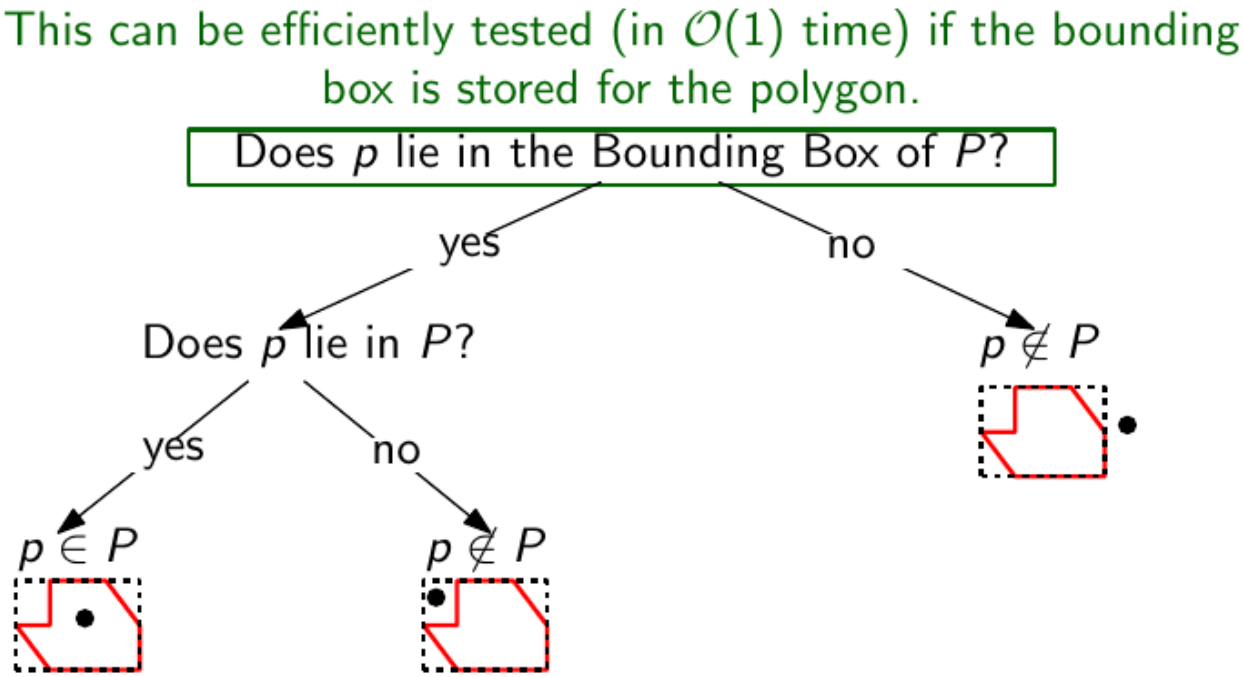
Difficulty: horizontal polygon edges

Solution: Treat polygon vertices on s as if they were above s





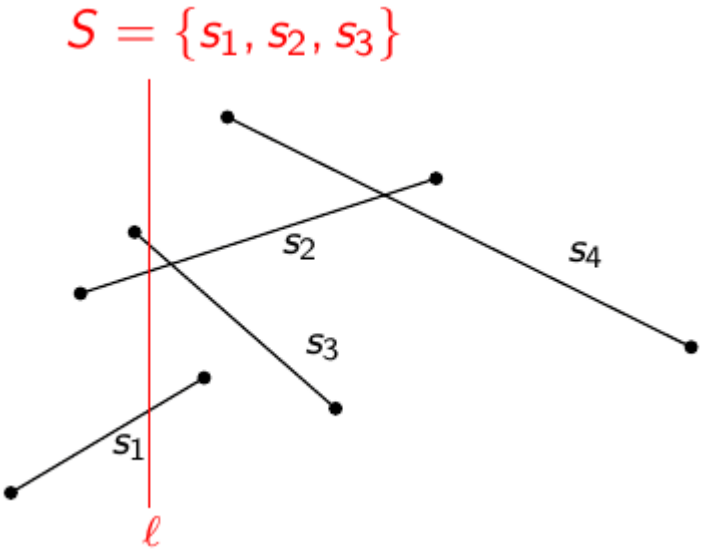
- In $\mathcal{O}(n)$, where $n = \#vertices$
- Filter steps to increase efficiency

- 

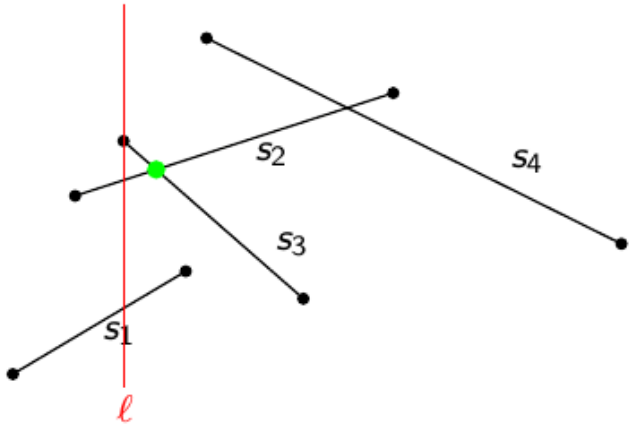
Intersection of Line Segments

- Given: Set of line segments in the plane
- Output: Intersections
- Limitation: Only consider intersections of the interiors of the line segments:
- Brute Force: $\mathcal{O}(n^2)$
  - Worst case: $n(n-1)/2$ intersections
  - -> No algorithm better than wc-runtime
  - ->
- **Output-Sensitive** algorithm
  - Runtime also depends on the output size
- Algorithm with runtime $\mathcal{O}((n+k)\log n)$
  - n = number of line segments
  - k = number of intersections
- For $k = n(n-1)/2$, the runtime is $\mathcal{O}(n^2 \log n)$, which is slightly *worse* than brute force
- BUT: If k intersection points is small, the algorithm is *much* better than brute force.

Basic Idea



- Vertical Line l sweeps the plane from left to right.
- During sweep: Maintain set S of all line segments that are intersected by l.
- S updated, when l reaches the left or right vertex of a line segment. Those points are termed events
  - e.g with a class with attributes Point, LineSegment, Type (string)
  - Set up the Events for every Line Segment
  - Go through all events and keep S up to date
- Brute force, thus $\mathcal{O}(|S|)$ runtime



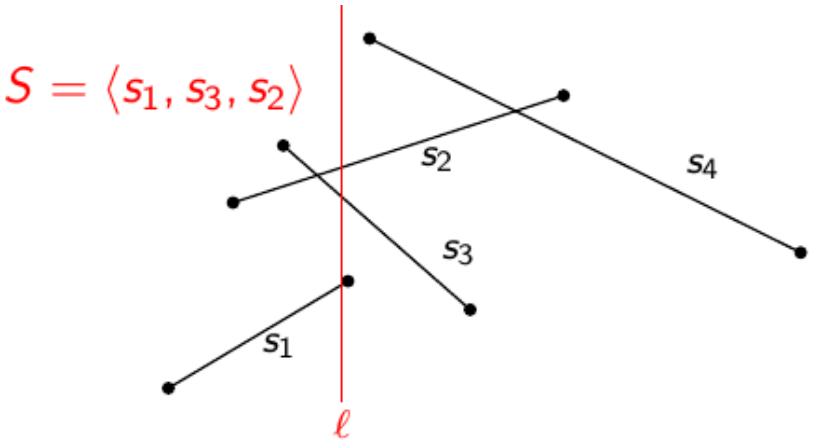
events:
$s_1.first$
$s_2.first$
$s_3.first$
$s_1.second$
$s_4.first$
$s_3.second$
$s_2.second$
$s_4.second$

before event:
$S = \{s_1, s_2\}$

testing for intersections:
$s_3 \cap s_1 = \emptyset$, $s_3 \cap s_2 = p_1$

after event:
$S = \{s_1, s_2, s_3\}$

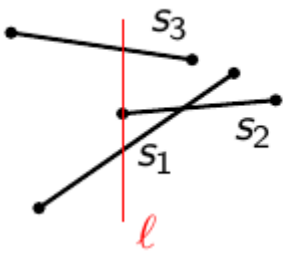
- Algorithm tests line segments, that overlap in x-dimension
  - If all overlap, all pairs of lines are tetsted
- -> Runtime of $\mathcal{O}((n+k)\log n)$ not yet reached



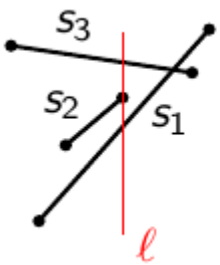
- Vertical line ℓ sweeps the plane from left to right.
- During this process we maintain the *sequence* S that contains all line segments intersected by ℓ, *ordered from bottom to top*.
- S needs to be updated at certain *event points*:

  begin and end of a line segment as well as *intersection points*

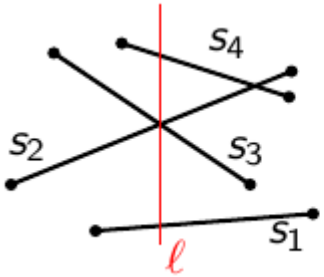
-
  - THEN: Use binary search tree for S in $\mathcal{O}(\log |S|)$ for each event
- Registation of Intersection events:
  - When line segment s_1 in S gets new neighbor s_2 (only happens at an event point)
  - s_1 and s_2 are tested for intersection
    - new intersection event added to the sequence of event
- $2n + k$ events that have to be handled
- Sequence of events E in binary Tree (processing in correct order) ->
  - $\mathcal{O}((n+k)\log(n+k))$ time of insertions and deletions
  - Status S contains at most n lines segments und is updated $2(n+k)$ times-> $\mathcal{O}((n+k)\log n)$ time
  - Per event: at most 2 intersection are computed

- 

  - s_2 is inserted in S.
  - s_2 is tested with s_1 for intersection $\rightarrow$ new event
  - s_2 is tested with s_3 for intersection $\rightarrow$ no new event

- 

  – s_2 is removed from S.
  – s_1 is tested with s_3 for intersection $\rightarrow$ intersection found, but no new event, since known already. (s_1 and s_3 were neighbors in S before.)

- 

  – s_2 and s_3 are swapped in S.
  – s_2 is tested with s_4 for intersection $\rightarrow$ new event
  – s_3 is tested with s_1 for intersection $\rightarrow$ no new event

  - s_2 and s_3 are swapped in S.

- Total runtime: $\mathcal{O}((n+k)\log(n+k)) = \mathcal{O}((n+k)\log n)$

Summary

- **Output sensitive**
- **Sweep-line pardigm** - quite common in computational geometry
- 2 **balanced binary search trees**
  - (one for the event sequence and one for the status sequence)